

ARTI 502

Deep Learning: 1st Assignment

Training-Testing Neural Net with MNIST Dataset using Pytorch

Year 2026 Term 1

Group 1 – 9FA01S

Group Members:

#	Member Name	Member Academic ID
1	Fedaa Alsabea	2230003675
2	Hussa Al-Turki	2230007134
3	Maha Al-Mansoori	2230002711
4	Sara Al-Idrissi	2230004327
5	Rimas Alajmi	2230002688
6	Fatima Al- Shihab	2230002074

Overview

In this assignment, you will perform Training Neural Net on the MNIST Dataset dataset using Pytorch. The goal is to explore the Basic Neural Network, understand its structure, and identify important issues such as architecture, learning rate, momentum and other parameters. You will also analyze relationships between number of layers and number of neurons for each layers to the overall Neural Network performance through their tuning hyperparameters.

Objective

Explore the dataset using PyTorch to understand:

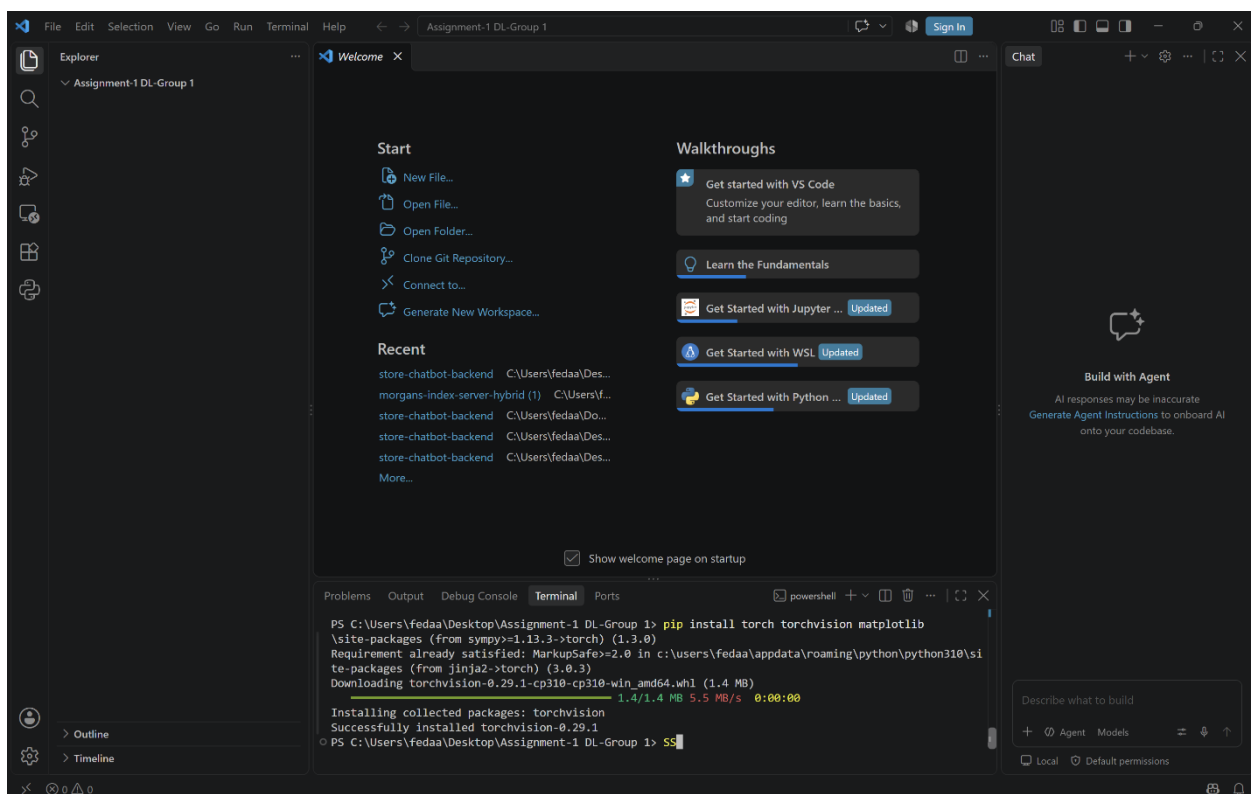
- Simple Neural Networks structure
- How to do Training-Testing Neural Networks
- How to evaluate the performance
- How to determine learning rate, momentum and model hyperparameters

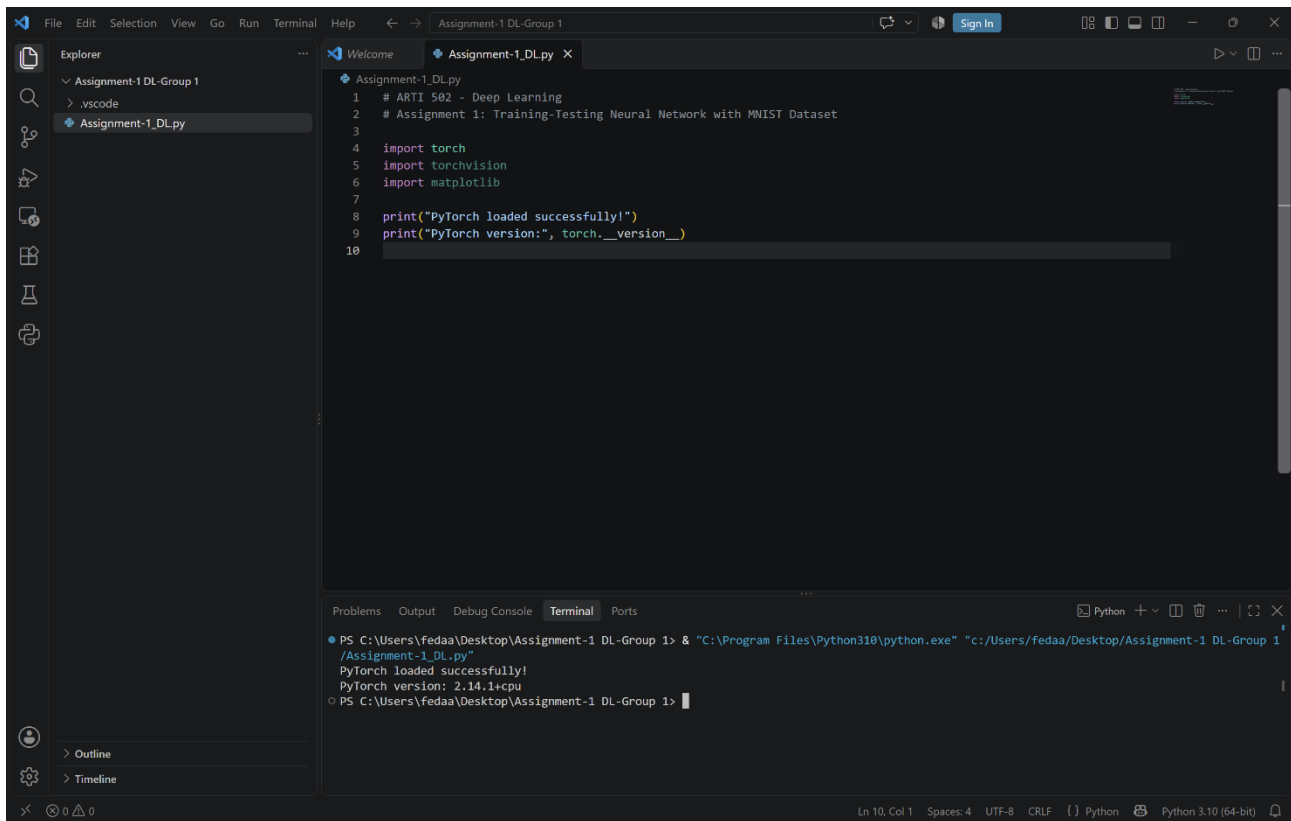
Tasks (with PyTorch)

Task 1: Install PyTorch

Instructions:

- Install PyTorch and load it to your python code and save it as ‘Assignment-1 DL’
- Screenshoot how your load PyTorch to your python source code.





Task 2: Download and Load the Dataset

Instructions:

- PyTorch has some built tools for downloading and loading common datasets. You will be playing around with MNIST in this assignment. It is a dataset of 28x28 grayscale handwritten digits 0-9. There are 50,000 images in the training set and 10,000 in the test set.
- Download the dataset directly from your python code then load it into the the workspace.

Deliverable:

- Screenshot showing the dataset loaded successfully.

```

14 from torchvision import datasets, transforms
15 from torch.utils.data import DataLoader
16
17 # Convert MNIST images to PyTorch tensors
18 transform = transforms.ToTensor()
19
20 # Download and load the training dataset
21 train_dataset = datasets.MNIST(
22     root="./data",
23     train=True,
24     download=True,
25     transform=transform
26 )
27
28 # Download and load the testing dataset
29 test_dataset = datasets.MNIST(
30     root="./data",
31     train=False,
32     download=True,
33     transform=transform
34 )
35
36 # Create data loaders
37 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
38 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

```

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1
/Assignment-1_DL.py"
PyTorch loaded successfully!
PyTorch version: 2.14.1+cpu
100%|#####| 9.91M/9.91M [00:01<00:00, 5.81MB/s]
100%|#####| 28.9k/28.9k [00:00<00:00, 182kB/s]
100%|#####| 1.65M/1.65M [00:01<00:00, 1.52MB/s]
100%|#####| 4.54k/4.54k [00:00<?, ?B/s]

MNIST dataset loaded successfully!
Training images: 60000
Testing images: 10000
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>

```

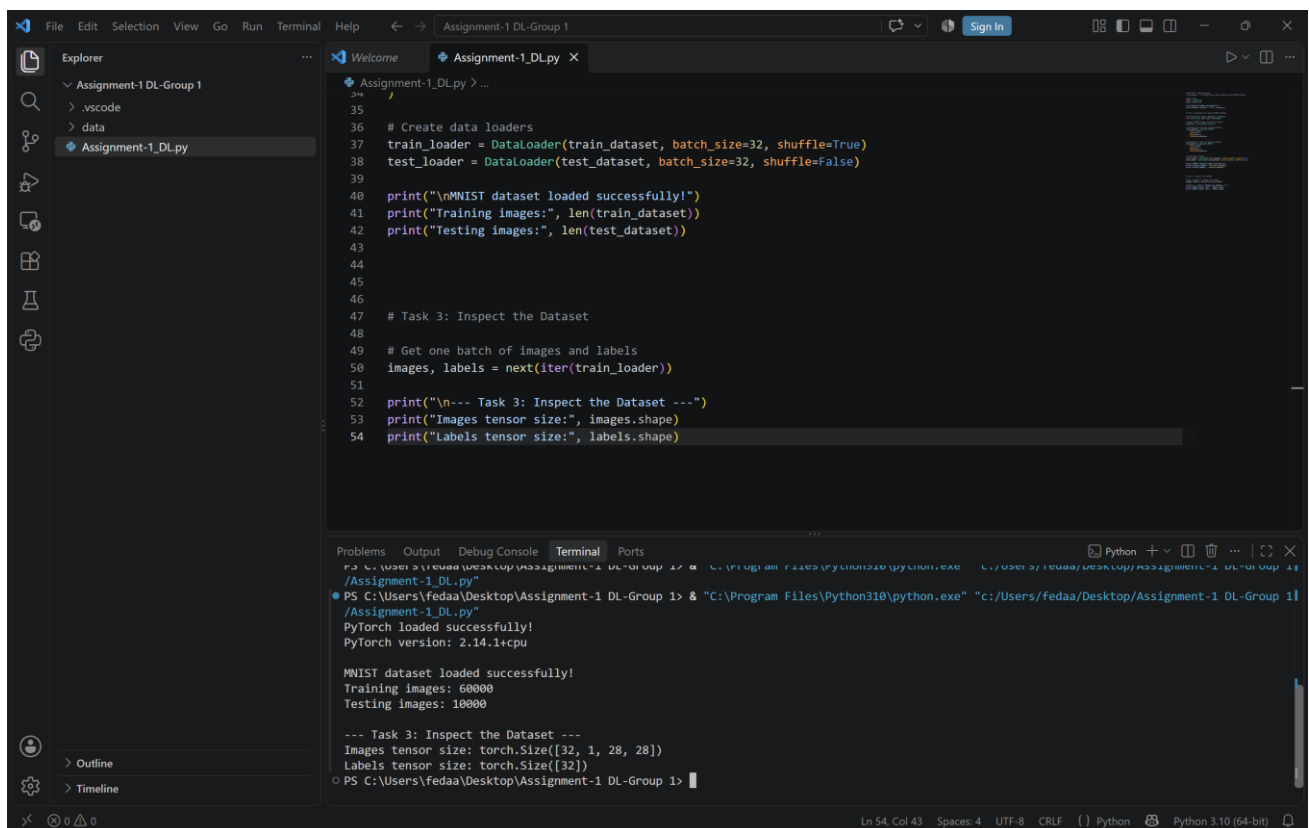
Task 3: Inspect the Dataset

Instructions:

- It's worthwhile to check out how the dataloader loads the images into tensors. You can print out the size of the loaded data in the images tensors. Also you can use matplotlib to see the visualization of the images data

Deliverable:

- Provide screenshots how to find:
 - Images tensor size
 - Data images vizualization using matplotlib



```

35
36 # Create data loaders
37 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
38 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
39
40 print("\nMNIST dataset loaded successfully!")
41 print("Training images:", len(train_dataset))
42 print("Testing images:", len(test_dataset))
43
44
45
46
47 # Task 3: Inspect the Dataset
48
49 # Get one batch of images and labels
50 images, labels = next(iter(train_loader))
51
52 print("\n--- Task 3: Inspect the Dataset ---")
53 print("Images tensor size:", images.shape)
54 print("Labels tensor size:", labels.shape)

```

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1/Assignment-1_DL.py"
PyTorch loaded successfully!
PyTorch version: 2.14.1+cpu

MNIST dataset loaded successfully!
Training images: 60000
Testing images: 10000

--- Task 3: Inspect the Dataset ---
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>

```



Figure 1

Sample Images from MNIST Dataset

Label: 5	Label: 5	Label: 6	Label: 1	Label: 4
Label: 9	Label: 3	Label: 3	Label: 4	Label: 1

```
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1/Assignment-1_DL.py"
PyTorch loaded successfully!
PyTorch version: 2.14.1+cpu

MNIST dataset loaded successfully!
Training images: 60000
Testing images: 10000

--- Task 3: Inspect the Dataset ---
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])
```

Task 4: Creating A Simple Network

Instructions:

- Write your Python code to setup the following simple network architecture:

First you'll build a very basic neural network with a single hidden layer of neurons. This means one fully connected layer of weights connects the input to the hidden neurons and one fully connected layer connects the hidden neurons to the output. Use the RELU activation function on the hidden neuron values as our nonlinearity.

These fully connected (or `nn.Linear`) layers expect a 2D input tensor where that is $N \times I$ where N is the number of data points in a **mini batch** and I is the **number of inputs**. However, our data is formatted in $(N \times C \times H \times W)$ right now so we need to tell PyTorch to rearrange it using `torch.flatten`. C , H and W are refer to Color, Height and Width respectively.

After being **flattened** our data goes from $(32 \times 1 \times 28 \times 28)$ to being (32×784)

Deliverables:

- Screenshot of source code
- Verify your code with above requirement.

```

76 class SimpleNeuralNetwork(nn.Module):
77     def forward(self, x):
78         # Hidden layer with RELU activation
79         x = F.relu(self.fc1(x))
80
81         # Output layer
82         x = self.fc2(x)
83
84         return x
85
86 # Create and verify the network
87 model = SimpleNeuralNetwork()
88
89 print("\n--- Task 4: Simple Neural Network ---")
90 print(model)
91
92 # Pass one batch through the network
93 sample_output = model(images)
94
95 print("\nInput images shape:", images.shape)
96 print("Output shape:", sample_output.shape)

```

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1
/Assignment-1_DL.py"
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])

--- Task 4: Simple Neural Network ---
SimpleNeuralNetwork(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

Input images shape: torch.Size([32, 1, 28, 28])
Output shape: torch.Size([32, 10])
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>

```

Task 5: Training Function

Instructions:

- Now for training the network. Your train function takes as input the net to train and the dataloader for the training data. It also takes some optional parameters to control training.
- For your network you will be using PyTorch's built in `nn.CrossEntropyLoss`. This will apply a softmax to our network's output, calculate the log-probability assigned to each class, then try to minimize the negative log likelihood of our data (AKA maximize the likelihood)
- For your optimizer you are using stochastic gradient descent with learning rate and momentum

Deliverables:

- Screenshot of your python source code that contains your traing function with above requirements.

```

111
112 # Task 5: Training Function
113
114 import torch.optim as optim
115
116 def train_network(net, train_loader, epochs=5, learning_rate=0.01, momentum=0.9):
117
118     # Loss function
119     criterion = nn.CrossEntropyLoss()
120
121     # SGD optimizer with learning rate and momentum
122     optimizer = optim.SGD(
123         net.parameters(),
124         lr=learning_rate,
125         momentum=momentum
126     )
127
128     losses = []
129     accuracies = []
130
131     for epoch in range(epochs):
132         running_loss = 0.0
133         correct = 0
134         total = 0
135
136         net.train()
  
```

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1/Assignment-1_DL.py"
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])

--- Task 4: Simple Neural Network ---
SimpleNeuralNetwork(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

Input images shape: torch.Size([32, 1, 28, 28])
Output shape: torch.Size([32, 10])
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>
  
```

Task 6: Training Network

Instructions:

- You will instantiate a new network and train it on your training data.
- Your training function prints out some debug information about the epoch, batch number, and current loss and accuracy values. It also returns a list of all the losses on your mini-batches so you can plot them all once training has finished.

Deliverables:

- Screenshot of instantiate training source code
- Screenshot of training debug which include epoch, batch number and current loss, accuracy values and graph for overall processes (loss and accuracy for each epoch).

The screenshot shows a VS Code editor with a file named `Assignment-1_DL.py` open. The code defines a `train_network` function that takes a network, a train loader, and training parameters (epochs=5, learning_rate=0.01, momentum=0.9). The function prints debug information for each epoch and batch, including current loss and accuracy. The terminal output shows the execution of the script, displaying the training progress for 5 epochs and 1875 samples per epoch.

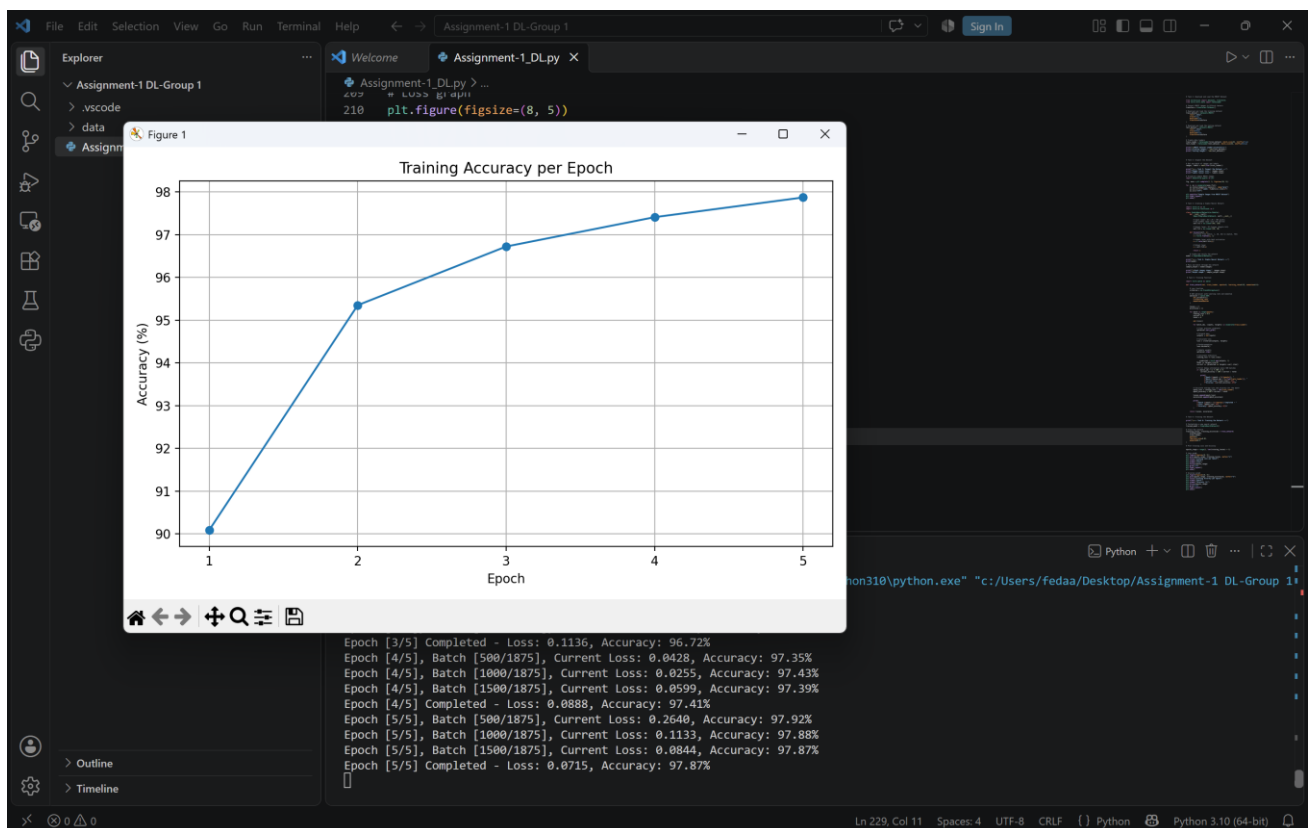
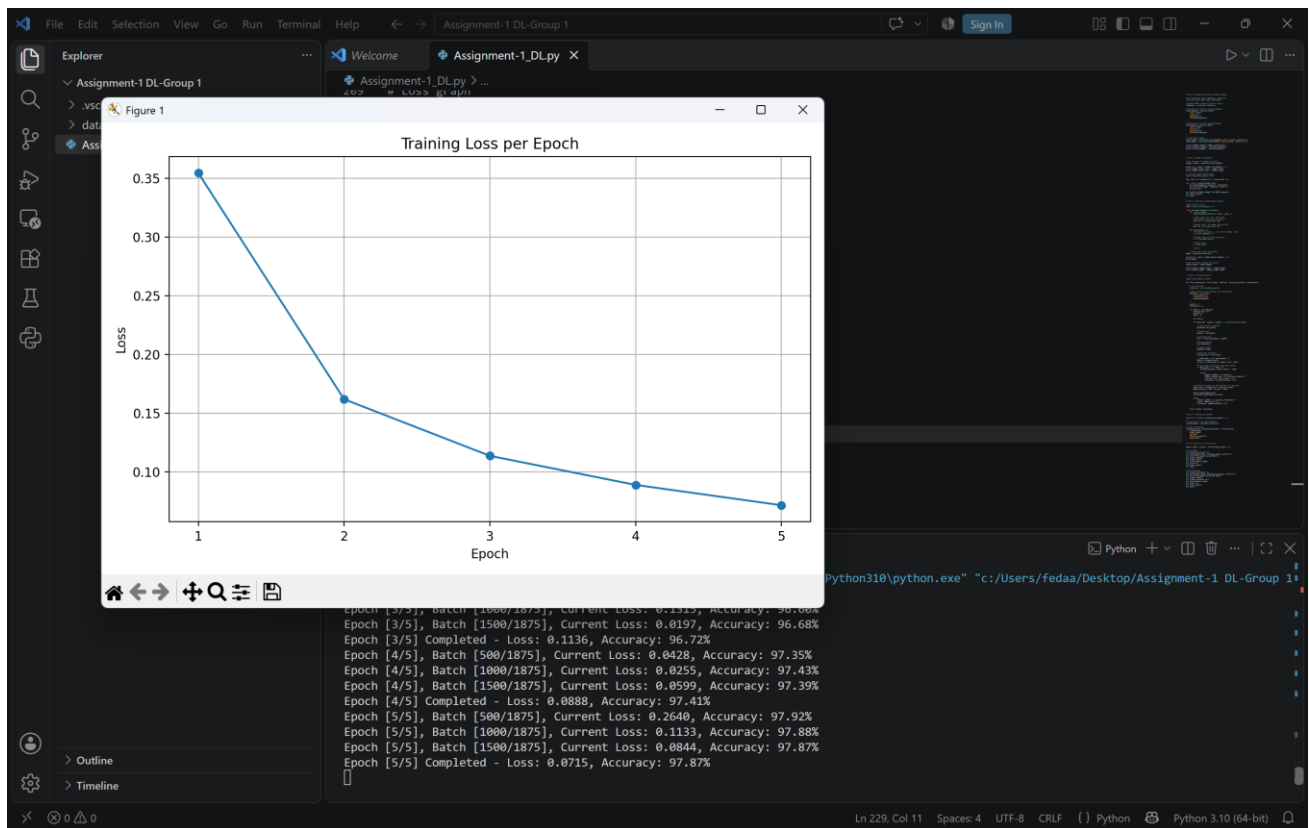
```
def train_network(net, train_loader, epochs=5, learning_rate=0.01, momentum=0.9):
    return losses, accuracies

# Task 6: Training the Network
print("\n--- Task 6: Training the Network ---")

# Instantiate a new neural network
trained_model = SimpleNeuralNetwork()

# Train the network
training_losses, training_accuracies = train_network(
    trained_model,
    train_loader,
    epochs=5,
    learning_rate=0.01,
    momentum=0.9
)
```

```
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:\Users\fedaa\Desktop\Assignment-1 DL-Group 1\Assignment-1_DL.py"
Epoch [3/5], Batch [1000/1875], Current Loss: 0.0557, Accuracy: 96.51%
Epoch [3/5], Batch [1500/1875], Current Loss: 0.0223, Accuracy: 96.56%
Epoch [3/5] Completed - Loss: 0.1142, Accuracy: 96.67%
Epoch [4/5], Batch [500/1875], Current Loss: 0.0098, Accuracy: 97.45%
Epoch [4/5], Batch [1000/1875], Current Loss: 0.0232, Accuracy: 97.37%
Epoch [4/5], Batch [1500/1875], Current Loss: 0.1146, Accuracy: 97.34%
Epoch [4/5] Completed - Loss: 0.0895, Accuracy: 97.41%
Epoch [5/5], Batch [500/1875], Current Loss: 0.1050, Accuracy: 97.83%
Epoch [5/5], Batch [1000/1875], Current Loss: 0.1445, Accuracy: 97.73%
Epoch [5/5], Batch [1500/1875], Current Loss: 0.0764, Accuracy: 97.83%
Epoch [5/5] Completed - Loss: 0.0736, Accuracy: 97.81%
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>
```



Task 7: Testing Network

Instructions:

- You can test your network on a few of your test images from task 2 and see what happens.

Deliverables:

- Screenshot your network testing source code
- Screenshot the results of the network testing

The screenshot shows a VS Code editor with a file named 'Assignment-1_DL.py'. The script is for testing a trained model. It includes comments in Arabic and Python code that calculates test accuracy. The terminal output shows the results of the training and testing process.

```

242 # task 7: testing the Network
243
244 print("\n--- Task 7: Testing the Network ---")
245
246 trained_model.eval()
247
248 correct = 0
249 total = 0
250
251 # Disable gradient calculation during testing
252 with torch.no_grad():
253     for images_test, labels_test in test_loader:
254
255         outputs = trained_model(images_test)
256
257         _, predicted = torch.max(outputs, 1)
258
259         total += labels_test.size(0)
260         correct += (predicted == labels_test).sum().item()
261
262 # Calculate overall test accuracy
263 test_accuracy = 100 * correct / total
264
265 print(f"Correct predictions: {correct}/{total}")
266 print(f"Test Accuracy: {test_accuracy:.2f}%")
  
```

Terminal Output:

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1
/Assignment-1_DL.py"
Epoch [4/5], Batch [1000/1875], Current Loss: 0.0741, Accuracy: 97.28%
Epoch [4/5], Batch [1500/1875], Current Loss: 0.0593, Accuracy: 97.28%
Epoch [4/5] Completed - Loss: 0.0915, Accuracy: 97.34%
Epoch [5/5], Batch [500/1875], Current Loss: 0.0501, Accuracy: 97.90%
Epoch [5/5], Batch [1000/1875], Current Loss: 0.0840, Accuracy: 97.87%
Epoch [5/5], Batch [1500/1875], Current Loss: 0.2307, Accuracy: 97.86%
Epoch [5/5] Completed - Loss: 0.0748, Accuracy: 97.84%

--- Task 7: Testing the Network ---
Correct predictions: 9718/10000
Test Accuracy: 97.18%
  
```

Task 8: Tuning the parameters and model architecture

Instructions:

- Tune the parameters with different values of learning rate and momentum and write your best results (lowest loss and highest accuracy).
- Tune the architecture model with different values of number of layer and number of neuron and write your best results (lowest loss and highest accuracy).

Deliverables:

- Screen your source python code with the best results.

```

260 # Task 8: Tuning Parameters and Model Architecture
261
262 print("\n--- Task 8: Parameter Tuning ---")
263
264 # Different learning rate and momentum combinations
265 parameter_settings = [
266     (0.01, 0.9),
267     (0.001, 0.9),
268     (0.01, 0.5)
269 ]
270
271 tuning_results = []
272
273 for lr, mom in parameter_settings:
274     print(f"\nTesting Learning Rate = {lr}, Momentum = {mom}")
275
276     # Create a new model for each experiment
277     tuning_model = SimpleNeuralNetwork()
278
279     # Train the model
280     losses, accuracies = train_network(
281         tuning_model,
282         train_loader,
283         epochs=3,

```

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:\Users\fedaa\Desktop\Assignment-1 DL-Group 1\Assignment-1_DL.py"
Epoch [2/3] Completed - Loss: 0.3084, Accuracy: 91.19%
Epoch [3/3], Batch [500/1875], Current Loss: 0.4604, Accuracy: 92.22%
Epoch [3/3], Batch [1000/1875], Current Loss: 0.1694, Accuracy: 92.28%
Epoch [3/3], Batch [1500/1875], Current Loss: 0.5111, Accuracy: 92.44%
Epoch [3/3] Completed - Loss: 0.2595, Accuracy: 92.57%
Result -> Learning Rate: 0.01, Momentum: 0.5, Test Accuracy: 93.65%

--- Parameter Tuning Summary ---
Learning Rate = 0.01, Momentum = 0.9 -> Test Accuracy = 96.81%
Learning Rate = 0.001, Momentum = 0.9 -> Test Accuracy = 91.69%
Learning Rate = 0.01, Momentum = 0.5 -> Test Accuracy = 93.65%
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>

```



The image shows a Visual Studio Code editor window with a Python file named 'Assignment-1_DL.py'. The script defines a 'TunedNeuralNetwork' class with two hidden layers and an output layer. It also includes a 'forward' method. The terminal output shows the execution of the script, displaying training progress for three epochs and the final architecture tuning result.

```

323
324 # Task 8 - Part 2: Model Architecture Tuning
325
326 print("\n--- Task 8: Model Architecture Tuning ---")
327
328 # Model with two hidden layers
329 class TunedNeuralNetwork(nn.Module):
330
331     def __init__(self):
332         super(TunedNeuralNetwork, self).__init__()
333
334         # First hidden layer: 784 -> 256 nodes
335         self.fc1 = nn.Linear(784, 256)
336
337         # Second hidden layer: 256 -> 128 nodes
338         self.fc2 = nn.Linear(256, 128)
339
340         # Output layer: 128 -> 10 classes
341         self.fc3 = nn.Linear(128, 10)
342
343     def forward(self, x):
344
345         # Flatten the image
346         x = torch.flatten(x, 1)
347

```

Terminal Output:

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1/Assignment-1_DL.py"
Epoch [2/3], Batch [1500/1875], Current Loss: 0.0186, Accuracy: 96.07%
Epoch [2/3] Completed - Loss: 0.1280, Accuracy: 96.16%
Epoch [3/3], Batch [500/1875], Current Loss: 0.0549, Accuracy: 97.38%
Epoch [3/3], Batch [1000/1875], Current Loss: 0.0285, Accuracy: 97.40%
Epoch [3/3], Batch [1500/1875], Current Loss: 0.0612, Accuracy: 97.37%
Epoch [3/3] Completed - Loss: 0.0856, Accuracy: 97.44%

--- Architecture Tuning Result ---
Original Architecture: 784 -> 128 -> 10
Tuned Architecture: 784 -> 256 -> 128 -> 10
Tuned Architecture Test Accuracy: 97.34%
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>

```

Task 9: Save your source code

Instructions:

- Save your python source code

Deliverable:

- Submit your Github link of your source code along with this document, which includes all screenshots via Blackboard.

Github link:

<https://github.com/fedaaalsabea/Deep-Learning-Assignment-1-Group1>

- End of Assignment -

Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Install PyTorch	PyTorch installed successfully and PyTorch loaded correctly with clear structure	Installed and PyTorch loaded with minor issues	pyTorch exists but unclear or incomplete	Not completed / missing	0.5
Task 2: Download and Load the Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	0.5
Task 3: Inspect the Dataset	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	0.5
Task 4: Creating A Simple Network	A simple network correctly created, screenshot included	Minor reporting issues	Node used but missing result explanation or screenshot	Not done	2
Task 6: Training Function	Training function configured correctly with clear screenshots of output	Minor reporting issues	Training Function used but missing result explanation or screenshot	Not done	1
Task 7: Testing Network	Correct Testing network created, screenshot included.	Testing network correct but result description is weak	Testing network exists but unclear results description	Not done	1



Task 8: Tuning the parameters and model architecture	Tuning parameters and model architecture created, screenshot included, and results description explained accurately	Tuning Parameters and model architecture correct but results description is weak	Tuning Parameters and model architecture correct but incorrect results description	Not done	1.5
Task 9: Save your source code	Source code save successfully	Source code save successfully done but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	0.5
Total					7.5